# Lecture 18 → Transport Layer, UDP, and Reliable Data Transfer

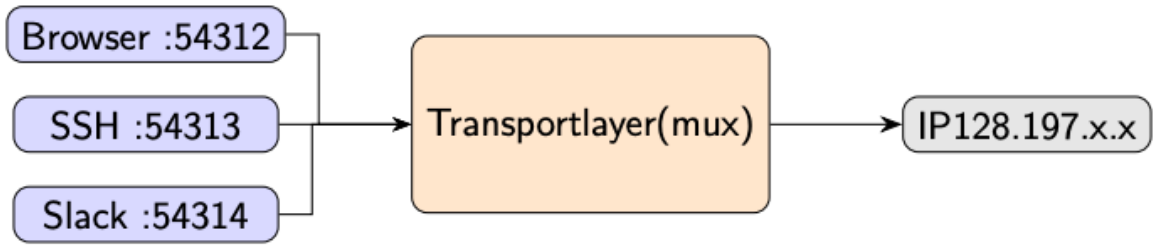
| Acronym | Meaning |
|---|---|
| ACK | Acknowledgment |
| ARQ | Automatic Repeat reQuest |
| BDP | Bandwidth-Delay Product |
| GBN | Go-Back-N |
| MSS | Maximum Segment Size |

| Acronym | Meaning |
|---|---|
| RDT | Reliable Data Transfer |
| RTT | Round-Trip Time |
| SR | Selective Repeat |
| UDP | User Datagram Protocol |
| S&W | Stop-and-Wait |

**From the Network Layer to the Transport Layer**
Network layer (IP) → delivers a datagram from host to host → best effort/ no guarantees
Transport layer (TCP/UDP) → delivers data from process to process → with optional reliability

| Physical | Link | Network | **Transport** | Application |
|---|---|---|---|---|
| bits | frames | datagrams | **segments** | messages |

**Multiplexing and Demultiplexing** → a single host runs many networked processes simultaneously, all sharing one IP address, ports are 16 bit numbers that identify endpoints within a host
Muliplexing (sender)→ collect data from multiple sockets and add ports headers and head to network layer
DeMuliplexing (receiver)→ read destination port and deliver each arriving segment to the correct sender




**Ports and Sockets**

Port ranges:

| Range | Name | Assigned by |
|---|---|---|
| 0–1023 | Well-known | IANA |
| 1024–49151 | Registered | Convention |
| 49152–65535 | Ephemeral | OS (client side) |

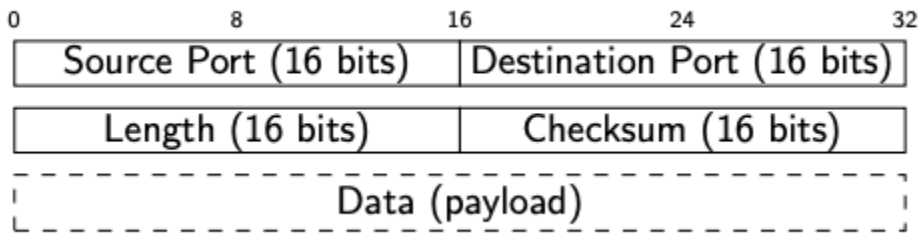
| Port | Proto | Service |
|---|---|---|
| 22 | TCP | SSH |
| 53 | UDP/TCP | DNS |
| 80 | TCP | HTTP |
| 443 | TCP | HTTPS |
| 67/68 | UDP | DHCP |

Socket → OS abstraction fro one communication endpoint
→ identified by 5 tuple:

1. Protocol (TCP or UDP)
2. Local IP address
3. Local port
4. Remote IP address
5. Remote port

UDP: demux key = (dst IP, dst port)
TCP: demux key = full 5-tuple

**UDP: The Minimal Transport Protocol** → UDP adds exactly 8 bytes of header → port numbers plus a checksum




- No connection setup
- No delivery guarantee
- No ordering guarantee
- No congestion control
- Yes port-based demultiplexing
- Yes optional checksum
- Yes immediate send (no handshake)
- Yes supports multicast/broadcast

**The UDP checksum**

- Ones' complement sum over a pseudo-header (source IP, destination IP, protocol = 17, UDP length) plus the UDP header and data

Why include IP addresses?

- Guards against misdelivered datagrams, a packet routed to the wrong host will fail the checksum even if the UDP header is intact
- Same principle as L17: ones' complement sum gives dmin = 2, detecting all single-bit errors

Optional in IPv4 (value 0x0000 = "not computed");

- mandatory in IPv6 (IPv6 dropped the IP-layer header checksum entirely)

UDP checksums are lightweight but not strong — no protection against reordering or multi bit errors. Higher-layer protocols (TLS, application) add stronger integrity when needed

**Why use UDP**

- Low latency interactive
  - VoIP, video calls, online games A dropped frame < a delayed frame
  - Retransmitting stale audio is useless
- Query response
  - DNS, NTP, DHCP
  - One datagram out, one datagram back
  - TCP setup adds ≥1.5 RTT for no benefit
- Multicast/ broadcast
  - UDP can address groups
  - TCP requires point-to-point connections
- Application controlled reliability
  - QUIC (HTTP/3), WebRTC App implements its own loss recovery, congestion control, and TLS on top of UDP, bypassing TCP's head-of-line blocking
- Tunneling and VPNs
  - WireGuard, DTLS Avoids "TCP-over-TCP" pathology: outer TCP retransmits interact badly with inner TCP congestion control

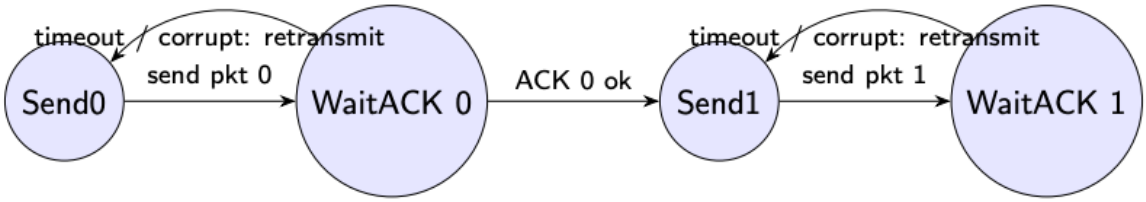
**The Reliable Data Transfer Problem** → the network layer delivers packets best effort → bits can be corrupted, packets can be lost → many applications cannot tolerate this

<u>Naiive answer:</u> just retransmit on failure

<u>The building blocks</u>:

- <u>sequence numbers</u>: label packets, detect gaps and duplicayes
- <u>Acknowledgements</u> (ACKs): receiver conforms correct receipts
- <u>Timers</u>: if no ACK within timeout → assume loss and retransmit
- <u>Checksum</u>: detect corrupted packets

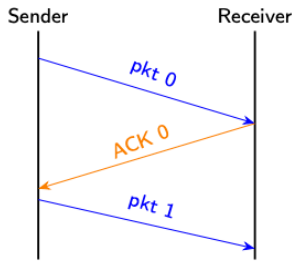
**Stop and Wait: what if the data packet is loss** → send one packet → wait for ACK → then send the next



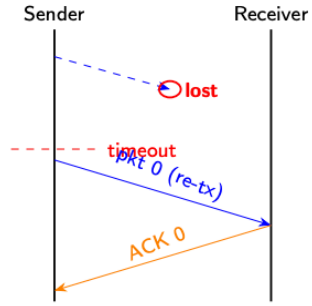

Only 1 bit sequence numbers needed → <u>alternating bot protocol</u>

**Stop and Wait: what if the ACK is lost** → data loss is handled cleanly: sender times out, retransmits, receiver ACKs. no ambiguity

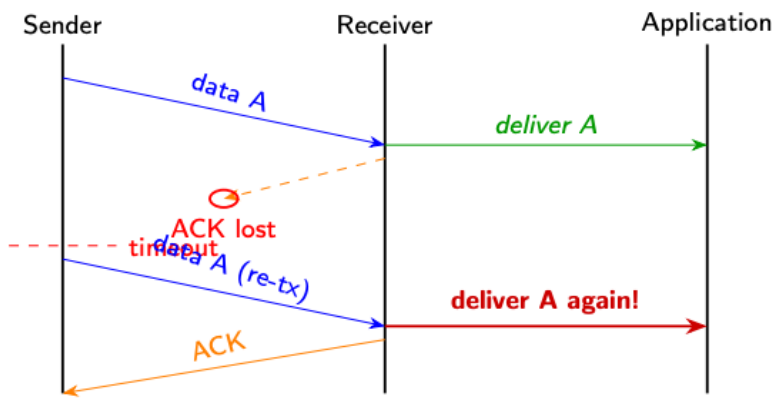
**Normal case** (no loss):




**Data packet lost**:




**Why sequence numbers are necessary** → Without sequence numbers: receiver cannot tell retransmit from new data




**Stop and wait: link utilization** → example: satellite link

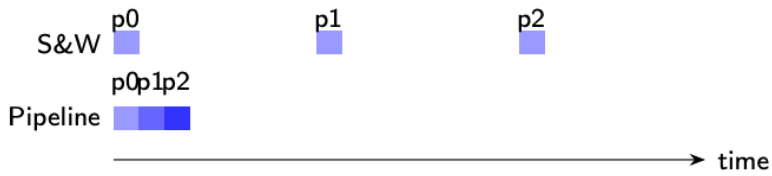
**Bandwidth delay product and pipelining (BDP)** → **t**he number of bits that could be in flight on a fully-utilized link

$$\text{BDP} = \text{Bandwidth} \times \text{RTT} = 1\ \text{Gb/s} \times 600\ \text{ms} = 600\ \text{Mb}$$

Stop-and-Wait keeps only one packet in flight (≈ 12,000 bits). To fill the pipe, we need:

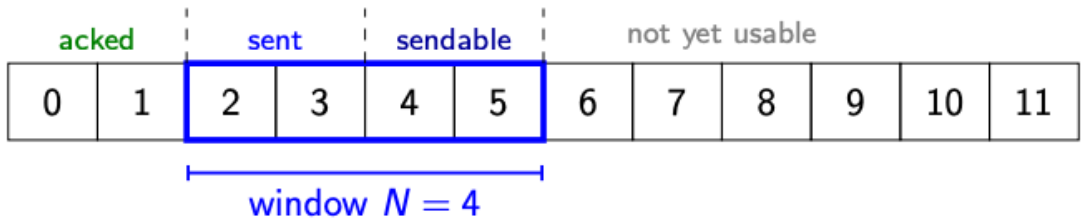
$$\text{Window size} \geq \frac{\text{BDP}}{\text{Packet size}} = \frac{600\ \text{Mb}}{12{,}000\ \text{bits}} \approx \mathbf{50{,}000}\ \text{packets}$$

<u>Pipelining</u>: send up to N packets before waiting for ACK → the window keeps the link busy




Two pipelining protocols: Go-Back-N and Selective Repeat → they differ in what happens

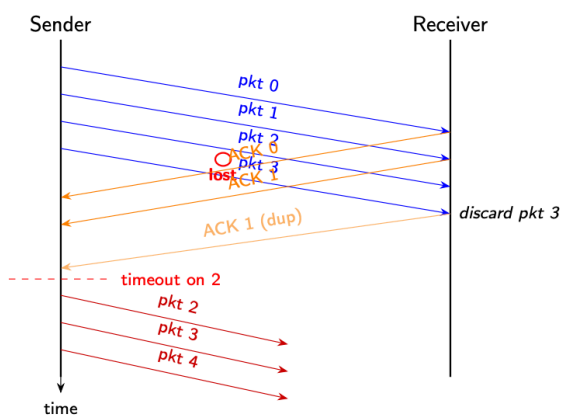
**Go Back N: sender window**




Rules :

- ACKs are cumulative: ACK n = ‘receoived all packets through n’
- One timer for the oldest unacknowledged packet
- On timeout: retransmit all packets in the window
- Receiver: out-of-order packets are discarded → no buffering needed
- Sequence numbers: window size N ≤ 2 k − 1 for k-bit sequence numbers

**Go Back N: packet loss trace** → One lost packet → retransmit all packets in the window.
Simple receiver; wasteful under loss
Window size N = 4. Packet 2 is lost:




**Selective repeat: Protocol** → **F**ix GBN’s inefficiency: the receiver buffers out-of-order packets; the sender retransmits only the lost packets

<u>Sender</u>:

- Same window concept as GBN
- Individual timer per packet
- On timeout for packet n: retransmit only packet n

- ACKs are individual (not cumulative)

<u>Receiver</u>:

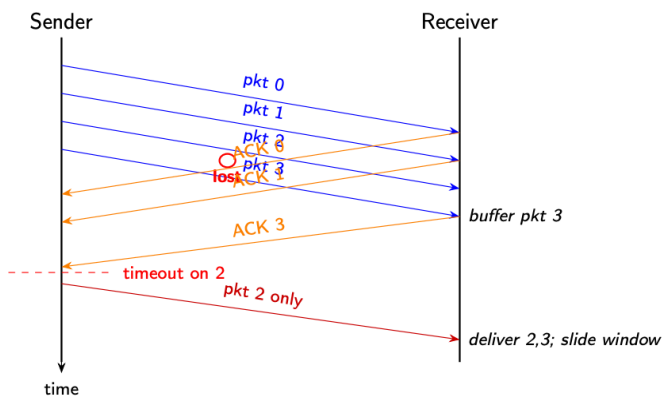
- If packet n is in [rcv base, rcv base+N-1]: buffer it, send ACK n
- If packet n = rcv base: deliver n + any consecutive buffered packets, slide window
- If duplicate of already-acked packet: send ACK n again (sender may not have received it

<u>Sequence number constraint:</u>

$N \leq 2^{k-1}$ (half the sequence number space)

**Selective repeat: same scenario**→ One lost packet → one retransmission. More complex receiver; much more efficient under loss

Window size N = 4. Packet 2 is lost:




**GBN vs SR: comparison**

| | **Go-Back-N** | **Selective Repeat** |
|---|---|---|
| Receiver buffer | None needed | Must buffer out-of-order pkts |
| Retransmit on loss | All packets in window | Only the lost packet |
| ACK type | Cumulative | Individual |
| Timers | One (oldest unacked) | One per unacked packet |
| Sequence number limit | $N \leq 2^k - 1$ | $N \leq 2^{k-1}$ |
| Good when | Loss rate is low | Loss rate is significant |

TCP borrows from both:

- Cumulative ACKs (GBN-style) → but also supports selective ACK (SACK) option
- Retransmits only missing segments when SACK is in use (SR-style)
- Receiver buffer required (like SR) → enables out-of-order delivery within the window